neatHack: the challenge and our solution

Working title: **Mend**, an agent that fixes itself, using neatlogs traces and Entire checkpoints. Prepared 9 October 2026, before kickoff. This is a plan: nothing described here has been built or measured yet.

1. The problem statement

neatHack has one track, **"Build an agent that works (literally)"**. Teams build an AI agent, or a team of agents, that completes a substantial task from start to finish: give it a goal, tools and an environment, and let each result guide its next action. The published focus areas are planning, tool use, state, context, execution, recovery, verification and iteration.

Every project must use three partner tools, each with a clear role:

- **neatlogs**: trace every agent run, find the code behind a failure, compare runs to verify a fix.
- **Entire**: repository context for coding agents; checkpoints that save the conversation behind each change, and entire-graph.
- **cfo.ai**: plan the agent's pricing, costs, revenue and runway. The business plan is part of the submission.

Key rules

Rule	What it says
Timing (IST)	Kickoff Sat 10 Oct, 12:30 PM. Submissions Mon 12 Oct, 7:00–11:55 PM. Winners 16 Oct.
Pre-event work	Planning is allowed. All code must be written during the hackathon; commits must show it.
Team	1–4 people, all registered on Luma under one team name.
Submission	Public quote post of the official neatlogs post on X, with team, project, public repo and demo link.
Video	3 minutes or less. Must cover neatlogs, Entire and cfo.ai: architecture, implementation, runtime, reasoning and result for each.

How projects are judged

Category	Weight	What it measures
The agent	30%	How well the agent handles a substantial task from start to finish
Use of neatlogs, Entire, cfo.ai	25%	How effectively the tools were used to build, improve and plan a business
Demo video	20%	How clearly the video explains the agent and the three tools
Building in public	15%	How the project was shared while it was built (#neatHack)
Usefulness and originality	10%	A problem people have, solved in a new way

Source: neatlogs.com/hackathon and luma.com/s5blr882, read 9 Oct 2026. Luma lists different event times (10 Oct 12:00 AM – 11 Oct 11:59 PM IST); to be confirmed on Discord.

2. The problem we are solving

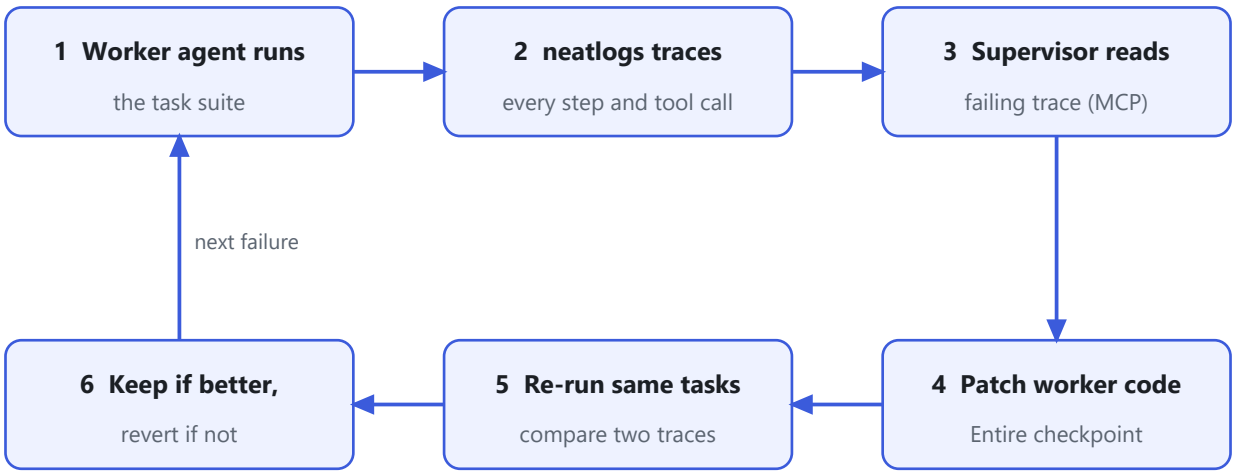
Agents fail quietly. A tool call returns something unexpected, the agent loses its state on a retry, or it declares success without checking. Today a human finds out later, opens the trace, works out which step went wrong, edits the agent's code or prompt, and re-runs it to see whether the change helped. That loop is slow, and the "did it actually help?" step is often skipped.

neatlogs already shows where a run failed, and Entire already records why code changed. Nobody is closing the loop automatically: failure, diagnosis, fix, and proof that the fix worked.

> **Mend is an agent that reads its own neatlogs traces, rewrites its own code, and proves the fix with a re-run.**

3. Our solution: how it works

Mend has two parts. A **worker agent** does a real task with an objective pass/fail result. Our default is fixing issues in a small sandbox repository, where "pass" means the repository's tests pass. A **supervisor agent** watches the worker and improves it.



- **Run.** The worker attempts a fixed set of tasks. Every model call, tool call and state change is traced in neatlogs.
- **Diagnose.** For each failed task, the supervisor pulls the trace through the neatlogs MCP server (the docs list `search_traces` and `get_trace_context`) and identifies the failing step.
- **Patch.** The supervisor changes the worker's code or prompt. Entire records the change as a checkpoint, together with the reasoning behind it.
- **Verify.** The worker re-runs the same tasks. The supervisor compares the before and after traces and keeps the change only if results improved; otherwise it reverts.

4. Role of each required tool

Tool	Role in Mend	Evidence it produces
neatlogs	Traces every worker run; the supervisor reads failing traces over MCP to diagnose; before/after traces verify each fix.	Trace links and screenshots; a failed run and what happened next; two traces with compared numbers

Tool	Role in Mend	Evidence it produces
Entire	Every self-applied patch becomes a checkpoint with the reasoning behind it, so each change the agent makes to itself can be audited and reverted.	Checkpoints and entire-graph output from our repository
cfo.ai	Plans Mend as a business: pricing per verified fix, model and infrastructure costs, revenue and runway scenarios.	The business plan included in the submission

5. Why this can win

Category (weight)	How Mend scores
The agent (30%)	Two agents completing a full loop: plan, use tools, keep state, recover, verify, iterate. It hits all eight published focus areas.
Tools (25%)	neatlogs and Entire are the core of the product, not add-ons. Four of the five judges come from these two companies.
Demo video (20%)	The story is built in: fail, trace, fix, proof. Every step is on screen.
Building in public (15%)	Each self-improvement round is a natural post: the bug, the trace, the fix, the new score.
Usefulness / originality (10%)	Every agent team has this problem; an agent that verifiably repairs itself is new.

The wow number

Tasks solved on a fixed task set before and after Mend repaired itself, for example “X of 20, then Y of 20”. We will only show a number we have actually measured.

The 30-second demo moment

The worker fails a task live. The supervisor highlights the failing step in the neatlogs trace. The patch appears as an Entire checkpoint with its reasoning. The re-run passes, and the two traces sit side by side.

Out of scope

- A polished product UI (a simple run viewer only)
- Changing real third-party repositories
- Automatic posting to social media

6. Risks and fallback

Risk	Mitigation
Two agents in 48 hours is a lot	Build the worker first. It is a valid submission on its own.
Self-repair may not actually improve results	Keep a fixed task set and report the real number, even if small. A manual trace-then-fix loop still demonstrates neatlogs and Entire.
neatlogs already offers fix suggestions (Investigations)	Position Mend as closing the loop: it applies the fix and proves it with a re-run.
neatlogs MCP and the Entire CLI not yet tried on our machine	Test both right after kickoff, before building on them. Entire installs via a bash script; may need Git Bash or WSL on Windows.

7. Plan for the weekend (IST)

When	Goal
Sat 10 Oct, from 12:30 PM	Repo created; neatlogs, Entire and model access tested; worker agent passing its first task with traces.
Sat night – Sun	Supervisor loop: read trace, patch, re-run, keep or revert. Build-in-public posts begin.
Sun	Fixed task set; measure before/after; cfo.ai business plan.
Mon 12 Oct	Generated demo video (3 min or less), README with evidence, quote post on X between 7:00 and 11:55 PM.

Still open

- Team members and skills; which model provider we use (AWS credits require early web check-in)
- Confirming the event times on Discord
- Final project name (“Mend” is a working title)